

Correction des exercices sur l'induction structurelle

Enseignement à distance de l'Université Marie et Louis Pasteur
Spécification et preuve de programmes

Version du 11 décembre 2025

Les exercices suivants demandent des démonstrations formelles, par induction structurelle, de propriétés de fonctions sur des types inductifs.

1 Type inductif des entiers naturels

1. La fonction $plus : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ qui réalise dans $\mathbb{N}$ l'addition de deux entiers naturels est définie par les égalités suivantes :

$$plus(x, 0) = x \quad (1)$$

$$plus(x, S(i)) = S(plus(x, i)) \quad (2)$$

Démonstration du lemme 1 par récurrence sur x . Soit $P(x)$ la formule $plus(0, x) = x$. On démontre séparément $P(0)$ et $\forall i. P(i) \Rightarrow P(S(i))$, comme suit.

Premier cas : $x = 0$.

$$\begin{aligned} & plus(0, 0) \\ = & \quad \quad \quad \text{d'après (1) où l'on remplace } x \text{ par } 0 \\ & 0 \end{aligned}$$

Deuxième cas : $x = S(i)$. L'hypothèse de récurrence est $P(i)$, c'est-à-dire $plus(0, i) = i$.

$$\begin{aligned} & plus(0, S(i)) \\ = & \quad \quad \quad \text{par (2) } (0/x) \\ & S(plus(0, i)) \\ = & \quad \quad \quad \text{par l'hypothèse de récurrence} \\ & S(i) \end{aligned}$$

2. Voici un codage de cette propriété comme un but WhyML.

```
goal plusZ: forall x. plus Z x = x
```

On ne peut pas le démontrer avec l'interface en ligne de Why3, car il faut indiquer qu'on veut procéder par récurrence sur x .

On peut le démontrer à l'aide de la fonction-lemme suivante :

```
let rec lemma plusZlf (x: nat): unit
  ensures { plus Z x = x }
= match x with
  | Z → ()
  | S m → plusZlf m
end
```

3. Démonstration de

$$plus(z, S(t)) = plus(S(z), t) \quad (3)$$

pour tous les entiers naturels z et t , par récurrence sur t .

Premier cas : $t = 0$.

$$\begin{aligned} & plus(z, S(0)) \\ = & \quad \text{d'après (2)} \\ & S(plus(z, 0)) \\ = & \quad \text{d'après (1)} \\ & S(z) \end{aligned}$$

et

$$\begin{aligned} & plus(S(z), 0) \\ = & \quad \text{d'après (1)} \\ & S(z). \end{aligned}$$

Deuxième cas : $t = S(i)$.

$$\begin{aligned} & plus(z, S(S(i))) \\ = & \quad \text{d'après (2)} \\ & S(plus(z, S(i))) \\ = & \quad \text{Hypothèse de récurrence} \\ & S(plus(S(z), i)) \end{aligned}$$

et

$$\begin{aligned} & plus(S(z), S(i)) \\ = & \quad \text{d'après (2)} \\ & S(plus(S(z), i)). \end{aligned}$$

4. Voici un codage de cette propriété comme un but WhyML.

```
goal plus_S: forall t. forall z. plus z (S t) = plus (S z) t
```

On ne peut pas le démontrer avec l'interface en ligne de Why3, car il faut indiquer qu'on veut procéder par récurrence sur t .

On peut le démontrer à l'aide de la fonction-lemme suivante :

```
let rec lemma plus_Slf (t: nat): unit
  ensures { forall z. plus z (S t) = plus (S z) t }
= match t with
  | Z -> ()
  | S m -> plus_Slf m
end
```

5. Démonstration de $plus(x, y) = plus(y, x)$ par récurrence sur y .

Premier cas : $y = 0$.

$$\begin{aligned} & plus(x, 0) \\ = & \quad \text{D'après (1)} \\ & x \end{aligned}$$

et

$$\begin{aligned}
& \text{plus}(0, x) \\
= & \quad \text{D'après le lemme 1} \\
& x.
\end{aligned}$$

Deuxième cas : $y = S(i)$.

$$\begin{aligned}
& \text{plus}(x, S(i)) \\
= & \quad \text{d'après (2)} \\
& S(\text{plus}(x, i))
\end{aligned}$$

et

$$\begin{aligned}
& \text{plus}(S(i), x) \\
= & \quad \text{D'après (3)} \\
& \text{plus}(i, S(x)) \\
= & \quad \text{D'après (2)} \\
& S(\text{plus}(i, x)) \\
= & \quad \text{Hypothèse de récurrence} \\
& S(\text{plus}(x, i)).
\end{aligned}$$

6. Voici un codage de cette propriété comme un but WhyML.

```
goal plusCommut: forall y. forall x. plus x y = plus y x
```

On ne peut pas le démontrer avec l'interface en ligne de Why3, car il faut indiquer qu'on veut procéder par récurrence sur y . On peut le démontrer à l'aide de la fonction-lemme suivante :

```
let rec lemma plus_Slf (t: nat): unit
  ensures { forall z. plus z (S t) = plus (S z) t }
= match t with
  | Z -> ()
  | S m -> plus_Slf m
end
```

7. La fonction *mult* qui réalise le produit de deux entiers naturels de $\mathbb{N}$ est définie par les égalités suivantes :

$$\text{mult}(x, 0) = 0 \quad (4)$$

$$\text{mult}(x, S(i)) = \text{plus}(\text{mult}(x, i), x) \quad (5)$$

8. Lemme 2 : $\text{plus}(a, \text{plus}(b, c)) = \text{plus}(\text{plus}(a, b), c)$ pour tous les entiers naturels a , b et c .
En admettant le lemme 2, démontrer la commutativité de *mult*, c'est-à-dire $\text{mult}(x, y) = \text{mult}(y, x)$, par récurrence sur y , puis, pour y fixé, par récurrence sur x .

— Premier cas : $y = 0$.

$$\begin{aligned}
& \text{mult}(x, 0) \\
= & \quad (4) \\
& 0
\end{aligned}$$

$$\begin{aligned}
& \text{— Premier cas : } x = 0. \\
& \quad \text{mult}(0, 0) \\
& \quad = \quad \text{par (4)} \\
& \quad 0 \\
& \text{— Deuxième cas : } x = S(z). \\
& \quad \text{mult}(0, S(z)) \\
& \quad = \quad (5) \\
& \quad \text{plus}(\text{mult}(0, z), 0) \\
& \quad = \quad (1) \\
& \quad \text{mult}(0, z) \\
& \quad = \quad \text{Hypothèse de récurrence} \\
& \quad 0
\end{aligned}$$

$$\begin{aligned}
& \text{— Deuxième cas : } y = S(t). \\
& \quad \text{mult}(x, S(t))
\end{aligned}$$

$$\begin{aligned}
& = \quad (5) \\
& \quad \text{plus}(\text{mult}(x, t), x)
\end{aligned}$$

Ensuite, il faut raisonner par récurrence sur x :

$$\text{— Premier cas : } x = 0.$$

Membre gauche	Membre droit
$\text{plus}(\text{mult}(0, t), 0)$	
$= \quad (1)$	
$\text{mult}(0, t)$	$\text{mult}(S(t), 0)$
$= \quad \text{Hypothèse de récurrence}$	$= \quad (4)$
$\text{mult}(t, 0)$	0
$= \quad (4)$	
0	

$$\text{— Deuxième cas : } x = S(z).$$

Le membre de gauche se simplifie comme suit :

$$\begin{aligned}
& \text{plus}(\text{mult}(S(z), t), S(z)) \\
& = \quad \text{Hypothèse de récurrence} \\
& \quad \text{plus}(\text{mult}(t, S(z)), S(z)) \\
& = \quad (4) \\
& \quad S(\text{plus}(\text{mult}(t, S(z)), z)) \\
& = \quad (5) \\
& \quad S(\text{plus}(\text{plus}(\text{mult}(t, z), t), z)) \\
& = \quad \text{Lemme 2} \\
& \quad S(\text{plus}(\text{mult}(t, z), \text{plus}(t, z)))
\end{aligned}$$

Le second membre se simplifie de la même manière (il suffit d'échanger z et t), donc

$$\begin{aligned}
& \text{mult}(S(t), S(z)) \\
& = \\
& \quad S(\text{plus}(\text{mult}(z, t), \text{plus}(z, t))) \\
& = \quad \text{Hypothèse de récurrence} \\
& \quad S(\text{plus}(\text{mult}(t, z), \text{plus}(z, t))) \\
& = \quad \text{Commutativité de plus} \\
& \quad S(\text{plus}(\text{mult}(t, z), \text{plus}(t, z)))
\end{aligned}$$

9. Démontrer

$$\text{mult}(s, \text{plus}(t, u)) = \text{plus}(\text{mult}(s, t), \text{mult}(s, u))$$

par récurrence sur s .

Premier cas : $s = 0$.

$$\begin{aligned}
& mult(0, plus(t, u)) \\
= & \quad \text{Commutativité de } mult \text{ (question 8)} \\
& mult(plus(t, u), 0) \\
= & \quad \text{D'après (4)} \\
& 0
\end{aligned}$$

et

$$\begin{aligned}
& plus(mult(0, t), mult(0, u)) \\
= & \quad \text{Commutativité de } mult \text{ (question 8)} \\
& plus(mult(t, 0), mult(0, u)) \\
= & \quad \text{Commutativité de } mult \text{ (question 8)} \\
& plus(mult(t, 0), mult(u, 0)) \\
= & \quad \text{D'après (4)} \\
& plus(0, mult(u, 0)) \\
= & \quad \text{D'après (4)} \\
& plus(0, 0) \\
= & \quad \text{D'après (1)} \\
& 0.
\end{aligned}$$

Deuxième cas : $s = S(v)$.

$$\begin{aligned}
& mult(S(v), plus(t, u)) \\
= & \quad \text{Commutativité de } mult \text{ (question 8)} \\
& mult(plus(t, u), S(v)) \\
= & \quad (5) \\
& plus(mult(plus(t, u), v), plus(t, u)) \\
= & \quad \text{Commutativité de } mult \text{ (question 8)} \\
& plus(mult(v, plus(t, u)), plus(t, u)) \\
= & \quad \text{Hypothèse de récurrence} \\
& plus(plus(mult(v, t), mult(v, u)), plus(t, u)) \\
= & \quad \text{Associativité de } plus \text{ (lemme 2)} \\
& plus(plus(plus(mult(v, t), mult(v, u)), t), u) \\
= & \quad \text{Associativité de } plus \text{ (lemme 2)} \\
& plus(plus(mult(v, t), plus(mult(v, u), t)), u)
\end{aligned}$$

et

$$\begin{aligned}
& plus(mult(S(v), t), mult(S(v), u)) \\
= & \text{Commutativité de } mult \text{ (question 8)} \\
& plus(mult(t, S(v)), mult(S(v), u)) \\
= & \text{Commutativité de } mult \text{ (question 8)} \\
& plus(mult(t, S(v)), mult(u, S(v))) \\
= & (5) \\
& plus(plus(mult(t, v), t), mult(u, S(v))) \\
= & (5) \\
& plus(plus(plus(mult(t, v), t), plus(mult(u, v), u)) \\
= & \text{Associativité de } plus \text{ (lemme 2)} \\
& plus(plus(plus(mult(t, v), t), mult(u, v)), u) \\
= & \text{Commutativité de } mult \\
& plus(plus(plus(mult(v, t), t), mult(u, v)), u) \\
= & \text{Commutativité de } mult \\
& plus(plus(plus(mult(v, t), t), mult(v, u)), u) \\
= & \text{Associativité de } plus \text{ (lemme 2)} \\
& plus(plus(mult(v, t), plus(t, mult(v, u))), u) \\
= & \text{Commutativité de } plus \\
& plus(plus(mult(v, t), plus(mult(v, u), t)), u)
\end{aligned}$$

2 Listes et arbres binaires

1. Type : $liste\text{-}vers\text{-}arbre : liste(\alpha) - \{Nil\} \rightarrow arbre(\alpha)$.

Égalités :

$$\begin{aligned}
liste\text{-}vers\text{-}arbre(Cons(x, Nil)) &= Leaf(x) & (6) \\
liste\text{-}vers\text{-}arbre(Cons(x, l)) &= Node(Leaf(x), liste\text{-}vers\text{-}arbre(l)) & \text{si } l \neq Nil(7)
\end{aligned}$$

2. Sous le nom ldf , la fonction $liste\text{-}des\text{-}feuilles$ est définie dans le cours.

Cas de base : $l = Cons(x, Nil)$ est une liste à un seul élément.

Alors,

$$\begin{aligned}
& ldf(liste\text{-}vers\text{-}arbre(Cons(x, Nil))) \\
= & \text{par (6)} \\
& ldf(Leaf(x)) \\
= & \text{par définition de } ldf \\
& Cons(x, Nil)
\end{aligned}$$

Pas d'induction :

Soit l une liste non vide ($l \in liste(\alpha) - \{Nil\}$). L'hypothèse d'induction est

$$ldf(liste\text{-}vers\text{-}arbre(l)) = l. \quad (8)$$

On a alors

$$\begin{aligned}
& \text{ldf}(\text{liste-vers-arbre}(\text{Cons}(x, l))) \\
= & \quad \text{par (7)} \\
& \text{ldf}(\text{Node}(\text{Leaf}(x), \text{liste-vers-arbre}(l))) \\
= & \quad \text{par définition de ldf} \\
& \text{append}(\text{ldf}(\text{Leaf}(x)), \text{ldf}(\text{liste-vers-arbre}(l))) \\
= & \quad \text{par définition de ldf} \\
& \text{append}(\text{Cons}(x, \text{Nil}), \text{ldf}(\text{liste-vers-arbre}(l))) \\
= & \quad \text{par l'hypothèse d'induction (8)} \\
& \text{append}(\text{Cons}(x, \text{Nil}), l) \\
= & \quad \text{par définition de append} \\
& \text{Cons}(x, \text{append}(\text{Nil}, l)) \\
= & \quad \text{par définition de append} \\
& \text{Cons}(x, l)
\end{aligned}$$

ce qui démontre le pas d'induction.

3 Preuve inductive sur les arbres binaires en WhyML

Le code suivant permet de prouver automatiquement la proposition

$$\text{Prop}(t) =_{\text{def}} \text{nb-feuilles}(t) = \text{length}(\text{ldf}(t))$$

pour tout arbre binaire t .

```

use int.Int

type bintree 'a = Leaf 'a | Node (bintree 'a) (bintree 'a)

let rec function nbLeaf (t : bintree 'a) : int
= match t with
| Leaf _ → 1
| Node t1 t2 → nbLeaf t1 + nbLeaf t2
end

use list.List
use list.Length
use list.Append

let rec function listOfLeaves (t : bintree 'a) : list 'a
= match t with
| Leaf a → Cons a Nil
| Node t1 t2 → listOfLeaves t1 ++ listOfLeaves t2
end

let rec lemma propLF (t : bintree 'a): unit
ensures { nbLeaf t = length (listOfLeaves t) }
= match t with
| Leaf _ → ()
| Node t1 t2 → propLF t1; propLF t2
end

```